SymCorr: A Compiler Framework for Automated Construction and Efficient Evaluation of Lattice QCD Correlation Functions

Tanishk Shrimal[a,*], M. Padmanath[a]

[a]The Institute of Mathematical Sciences Chennai, IV Cross Road, CIT Campus, Taramani, Chennai, 600113, Tamil Nadu, India

ABSTRACT

a

1. Introduction

The strong coupling of Quantum ChromoDynamics (QCD) at low energies precludes the use of perturbative approaches, where hadrons emerge as the relevant effective degrees of freedom. Consequently, hadron physics investigations in the low energy regime rely on nonperturbative approaches such as those using the lattice QCD framework. Generation of lattice QCD ensembles through Monte-Carlo simulations and estimation of fermion matrix determinant as well as inversions of the fermion matrix consume substantial fraction of the computer time in lattice QCD calculation and have been extensively discussed and investigated since the inception of lattice QCD []. Physical observables in hadron-physics investigations are extracted from Euclidean correlation functions. In the early years of lattice QCD, these calculations primarily targeted ground-state hadron properties and were comparatively inexpensive. The discovery and study of numerous heavy and excited hadrons over the past several decades, however, necessitated increasingly sophisticated spectroscopy calculations, notably those employing Euclidean correlation matrices and variational analyses to determine the finite-volume spectrum and its properties. As the size and complexity of these correlation matrices increase, the associated operator construction, Wick contractions, and numerical evaluation become increasingly demanding, resulting in substantial rise in computational costs.

Modern day hadron spectroscopy proceeds through evaluation of two point Euclidean correlation matrices

$$C_{ij}(t-t_0) = \left\langle \mathcal{O}_i(t)\, \mathcal{O}_j^{\dagger}(t_0) \right\rangle \tag{1}$$

for a basis of interpolators/operators $\{O_i\}$ that carry the quantum numbers of the desired hadronic state, where t and t_0 refers to the sink and the source timeslices. In this context, the new challenges include a systematic construction of interpolating operators that enter the evaluation of correlation matrices, handling the allowed Wick contractions for every combination of interpolating operators, keeping track of the fermionic signs with permutations, flavor, spin, color and spatial indices faithfully, etc. The amount of bookkeeping therefore grows as one moves from meson operators to baryons, multi-hadron operators and nuclei. Additional conversions and reductions need to be made to express the correlation matrices in terms of the objects defining operator structures as well as the fermion matrix inverses. Note that different matrix elements carry similarities and share quark-propagation data, operator-dependent building blocks, and often portions of the resulting contraction expressions. This observation suggests opportunities to identify repeated (sub)objects IO and computations, reducing the computational redundancy. While a systematic road-map has already been laid out for such excited hadron spectroscopic investigations in the literature, performing them manually for every new channel being studied requires substantial implementation effort and can introduce bookkeeping errors.

There already exists tools to handle the operator construction and correlation matrix evaluation. Tools such as Multi-Hadron Interpolators (MHI) [1] and `OpTion` [2] provides a systematic construction of multi-hadron lattice operators projected to finite-volume irreps, total lab frame momentum, spin, and internal quantum numbers. The QCT package [3] generates `C++` codes compatible with `QDP++` [4] to perform Wick contractions for a given set of hadronic currents/operators expressed in terms of (anti)quark fields, without further optimization for computational efficiency. The Redstar framework [5] expresses each element in correlation matrices as a sum of graphs (Wick contractions) and utilizes uses graph-contraction algorithms to reduce redundant evaluations and to organize optimized correlator evaluations in a distillation-based calculational setup [6] using tensor contractions for execution on CPUs and GPUs. Another recent algorithmic work explored determinant representations of Wick contractions and methods for identifying smaller sets of determinant terms that are sufficient for evaluating nuclear correlation functions [7].

The emphasis of the SymCorr package is on having a complete automated transformation from a symbolic quark-level operator description to a numerical calculation within the distillation prescription [6] as a sequence of explicit compilation stages. Once the user provides the symbolic definition of a basis of operators in terms of quark and gauge fields, the operator representations, Wick contractions, automated translation to backend-independent physics

*Corresponding author

tanishks@imsc.res.in (T. Shrimal); padmanath@imsc.res.in (M. Padmanath)

ORCID(s): 0000-0001-0000-0000 (T. Shrimal); 0000-0001-0000-0000 (M. Padmanath)

objects, distillation lowering, execution planning, and numerical evaluation are organized as distinct layers of the software. This staged organization reduces manual intervention in correlator-specific algebra and implementation, while maintaining a clear separation between the physical definition of an operator and the details of its numerical evaluation. It also makes the intermediate algebraic transformations explicit and amenable to inspection and validation, while allowing numerical execution strategies to evolve and adapt independent of the underlying physics description of the interpolating operators.

2. Overview and scope of SymCorr

SymCorr package facilitates the computation of Euclidean correlation matrices in the distillation framework, serving itself as a generalized automated platform in computation of arbitrary multi-hadron correlation matrices in finite-volume, with the assumption that the relevant lattice numerical data and the operator basis details are provided by the user. We begin this section by briefly discussing numerical objects relevant for the distillation framework for completeness. In the conventional non-distillation setup, the measurements involve computation of correlation matrices given in Eq. 1 for a given operator basis and varying values of t and t_0. These involve appropriate tensor index contractions between spatial-color-spin indices of the hadron interpolators with those in the quark line objects referred to as the quark propagators.

The computational cost of such tensor index contractions spanning through the space-color-spin index values ($\mathcal{O}(10^6)$ for a 40^3 spatial lattice) can become prohibitively expensive when then system being studied is large or complex such as multi-hadron operators, tetraquarks, pentaquarks, nuclei, etc. In the distillation framework, the combined space-color index is replaced by a single index, which could run over a appropriately chosen eigenmode subset of discretized gauge-covariant Laplacian $\{v^{(i)}(t)\}$. Such a replacement can be facilitated through quark smearing program defined in terms of $\{v^{(i)}(t)\}$ as

$$\begin{aligned} &\tilde{q}(t) = \square(t) q(t), \qquad \tilde{\bar{q}}(t) = \bar{q}(t) \square(t), \\ &\text{where} \quad \square(t) = V(t) V^\dagger(t) = \sum_{i=1}^{N_v} |v^{(i)}><v^{(i)}|(t), \\ &\text{and} \quad V(t) = \left(v^{(1)}(t), v^{(2)}(t), \dots, v^{(N_v)}(t)\right). \end{aligned} \tag{2}$$

The $\square(t)$ operator acts as a low pass filter or a projection operator, with the lowest N_v modes. Inserting such projection operators for the quark fields in the conventional correlations, followed regrouping the objects to form new objects such as

$$\tau_f(t, t_0) = V^\dagger(t) M_f^{-1}(t, t_0) V(t_0) \tag{3}$$

$$\Phi_{\mathcal{O}}(t) = V^\dagger(t)\, \mathcal{O}\, V(t). \tag{4}$$

referred to as the perambulators and elementals/kernels, which takes over the information from the quark propagator $M_f^{-1}(t, t_0)$ and hadronic operator $\Phi_A(t)$, respectively. The objects $\tau_f(t, t_0)$ and $\Phi_{\mathcal{O}}$ carry indices running of the $N_v \sim 100 - 1000$ modes considered facilitating contractions of tensor indices with reduced computational cost. This novel procedure was not only found to reduce the computational cost requirement involved in tensor contractions, but also found to provide access to all-to-all quark propagators, to source momentum projection in hadron operator, multi-hadron operators, etc. The expensive quark-propagation information can be prepared independently of a particular spectroscopy operator basis, while changes to momenta, spin structures, or spatial constructions modify the operator-dependent factors. Within the distillation framework, the correlation matrix elements can be written in a compact form as

$$C_{ij}(t, t_0) = \sum_\alpha c_\alpha \ \mathrm{Contr}_\alpha \left[\{\Phi\}, \{\tau\}\right], \tag{5}$$

where α runs over the Wick-contractions "Contr_α [...]" and c_α carries the associated numerical coefficient and the fermionic sign.

The SymCorr package is designed to separate the physical definition and representation of correlation measurements from the details of their numerical evaluation. The user-provided interpolators determine the Wick contractions and their representation in the distillation space, as illustrated schematically in Eq. 5, while the subsequent numerical procedures, including tensor contractions, redundancy optimizations to avoid repetitive evaluations, memory (de)allocation, and data reuse, are handled by the SymCorr backend. The physics-level description of the measurements are preserved at every stage of the SymCorr flowchart to facilitate user-end as well as backend verifications and troubleshooting, while the numerical tensor objects and operations remain blind to the physics that is aimed at. The correlation matrix evaluation is treated within the SymCorr package as a single structured computational problem, instead of compiling every matrix element as an independent numerical program. This helps in identifying similar algebraic expressions as well as redundant mathematical work across the full calculation and thus providing efficient choices for tensor contraction ordering and data reuse, enabling an optimized workflow. Following the optimization aprogram t the algebraic as well numerical level, SymCorr also makes educated decisions at the execution stage in regrouping calculations to utilize the provided the hardware resources to its maximum potential. All optimizations, at the algebraic, numerical, and execution stages, are handled by the SymCorr backend while preserving the underlying physics information. This shields non-expert users from the complexities of optimization, while still giving expert users full freedom to inspect the practical implementation and numerical performance.

We present the SymCorr workflow in Fig. 1. The user provides the interpolator basis in symbolic form (Sec. 3) together with a job specification that fixes the lattice dimensions, the number of distillation modes, the spin basis, the

data locations, and the source and sink times. The calculation then proceeds in two stages. The compile stage uses only the symbolic operators and the compile settings. Linear combinations of operators are expanded into basis correlators, and all quark-line pairings (Wick contractions), their fermionic signs, and their coefficients are generated for every element of the correlation matrix (Sec. 4). The resulting expressions are lowered to the distillation framework, where each Wick term becomes a tensor network of operator-dependent spatial kernels, propagation (perambulator) resources, and spin tensors (Sec. 5). All tensor networks of the requested correlation matrix are then treated as one global contraction problem: identical resources and identical contraction networks are shared across Wick terms and matrix elements, exactly equal terms are combined, a contraction order is chosen, shared intermediates are identified, and the operations are scheduled together with a plan for reusing memory buffers (Sec. 6). The result is saved as a portable plan that depends neither on lattice data nor on hardware. In the run stage, the saved plan is first checked against the current operator definitions and job specification. SymCorr then chooses a CPU or GPU deployment and a memory-compatible batching of time slices for the available hardware, reads the Laplacian eigenvectors $\{v^{(i)}(t)\}$, the perambulators $\tau_f(t, t_0)$ and, for derivative or displaced operators, the gauge configuration in the form of the stout-smeared spatial links used to construct the eigenvectors, builds the required kernels and propagation tensors, and executes the frozen plan for every gauge configuration and time context. The correlation functions are written to an HDF5 file; a record of the run and a human-readable report can be produced on request.

Now we present the scope of the SymCorr package. The present implementation can serve in optimized distillation-based correlation matrix evaluations and works on both CPU as well as GPU resources. The package assumes externally supplied operator basis, lattice QCD configuration, associated Laplacian eigenvectors, and the quark perambulators. Operator constructions such as those dealt by packages (MHI) and `OpTion` supplied n Ref. [1, 2], generation of the lattice QCD configurations [**?** **?**] and evaluation of Laplacian eigenvectors, and perambulators [**?**] are outside SymCorr's scope. The package can inspect the available resources such as CPUs, GPUs, memory, and can make educated decisions on efficient execution, however it does not support multi-GPU and MPI based multi-CPU evaluations, which is a straightforward extension. These are limited execution boundaries rather than restrictions on the symbolic definition of the correlator.

At the physics level, the SymCorr package has been validated for a variety of local as well as bilocal hadron operators, and their full distillation two point correlation matrix evaluations. The results has been cross-verified with independent Chroma based evaluations for rest as well as (multiple) moving frames, for single mesons, single baryons as well as bilocal two-hadron interpolators. Correlation matrices with local four-quark operator and relavant bilocal two meson operators in the T_{1g} have also been tested and cross-verified with an independent realization in the Croma software suite. Beyond this explicitly verified cases, the package has additional capabilities, that includes, but not limited to, multi-hadron correlation matrix evaluations, derivative-based single hadron correlator evaluations, correlation matrices involving disconnected quark loops, three point correlation matrices provided the generalized perambulators [].

3. Symbolic Representation of Interpolating Operators

The SymCorr package takes in the interpolator basis as a list of ordered algebraic expressions built out of objects representing (anti)quark fields, gauge fields, spin and color structures. At this stage of interpolator definition, the representation is independent of distillation tensors, data layouts, contraction paths, and execution backends. This strategy helps in retaining the physical structure at all stages of the execution.

Quark and antiquark fields of flavor f are represented schematically as

$$q_f(x)_{a\alpha}, \qquad \bar{q}_f(x)_{a\alpha}, \tag{6}$$

where x, a, and α denote position, color, and Dirac spin indices, respectively. The flavor and the distinction between a quark and an antiquark are set as the properties of the fermion field objects to facilitate easy identification of flavor compatibility and fermion permutation signs at the stage of Wick contractions.

The operator representations are independent of the Dirac gamma matrix conventions

Spin structure is represented by indexed matrices $\Gamma_{\alpha\beta}$, whose numerical convention is only assumed only at the backend.

The frontend provides the usual gamma matrices and also permits named spin matrices used by the supported operator constructions, including charge-conjugation structures. At the symbolic level the important information is the matrix identity together with its two spin indices; the concrete numerical representation of the spin matrices is a later convention and does not alter the operator expression.

Color contractions are explicit symbolic tensors. The current frontend contains both the Kronecker tensor δ_{ab} and the antisymmetric tensor ϵ_{abc}, which provide the basic color structures required by the implemented meson- and baryon-like constructions. Gauge transport can also be represented explicitly through a gauge-link object $U(x, y)_{ab}$ carrying its endpoints and open color indices. Keeping these objects visible in the expression prevents color connectivity from being inferred later from the ordering or shape of numerical arrays.

Spatial transformations acting on individual fields are represented separately from the fields themselves. The current frontend distinguishes a pure coordinate shift from gauge-covariant parallel transport, provides a symbolic covariant derivative along a directed lattice axis, and supports

field-level spatial projections such as the LapH and point representations. These transformations are attached to a particular field expression rather than being silently absorbed into the field. Consequently, an object such as $D_\mu q_f$ retains the fact that a derivative acts on that specific field as the expression proceeds through the compiler.

Products of the elementary objects form larger symbolic expressions, with the factor ordering retained explicitly. This is essential for fermionic expressions because the ordering of quark fields enters the sign obtained after Wick contraction. A complete interpolating operator then wraps such an expression together with its momentum assignment or assignments and optional metadata. For operators with several spatial locality groups, momentum can be associated with the corresponding position labels rather than being forced into one global momentum value.

A quark propagator is deliberately not part of the user-authored symbolic vocabulary. Propagator objects arise only after the source and sink expressions have been combined and Wick contraction has paired compatible quark and antiquark fields. This boundary ensures that the user specifies the interpolating operator itself, while propagation topology is derived from the field content rather than supplied manually.

3.1. Indices and quantum numbers

SymCorr assigns every symbolic index to one of three spaces: spin, color, or position. An index is therefore not identified by its text label alone. When a label is first introduced, its index space is fixed; attempting to reuse the same label as a different type is rejected. Reusing the label within the same space refers to the same symbolic index and hence defines the intended tensor connectivity of the operator.

Spin and color indices follow the usual Einstein-style interpretation within a symbolic expression. An index appearing twice denotes a contracted index, while a single occurrence remains open. Open spin indices are permitted because, for example, baryon operators can carry an uncontracted Dirac index. An uncontracted color index is allowed by the representation but is reported as a warning because the resulting operator may not be gauge invariant. Spin or color indices appearing more than twice are rejected rather than assigned an ambiguous contraction meaning.

Position indices play a different role. They identify the spatial or spacetime locality associated with fields and gauge structures and are therefore not subject to the same multiplicity rule. Two fields carrying the same position label belong to the same locality group, whereas distinct labels can be used to represent bilocal or more general composite operators. In this sense the position labels participate in the physical specification of the operator even though their literal names are dummy labels.

The frontend does not combine all quantum numbers into a single opaque tag. Flavor is stored directly on each quark field, spin and color information are encoded by the indexed fields and tensors, and momentum is attached to the operator through its position-locality groups. If an operator contains exactly one such group, a single three-component lattice momentum is unambiguous. If it contains several independent locality groups, the momentum assignment is given as a map from the corresponding position labels to three-component momenta,

$$x_r \longmapsto \mathbf{p}_r, \qquad \mathbf{P} = \sum_r \mathbf{p}_r, \tag{7}$$

where $\mathbf{P}$ is the total momentum carried by the composite operator. This representation keeps the momentum associated with each constituent locality visible rather than replacing it immediately by the total momentum. Under Hermitian conjugation, every stored momentum assignment changes sign, consistent with the reversed source–sink orientation.

The literal names chosen for dummy spin, color, and position indices are not part of the mathematical identity of an operator. SymCorr compares operator structure after a deterministic relabeling of these indices while preserving the ordered expression and the associated momentum assignments. Thus two operators that differ only by dummy-index names can be recognized as the same symbolic structure, whereas a change of flavor, tensor connectivity, factor ordering, spatial transformation, or momentum remains physically significant.

A second use of relabeling occurs when several operators are inserted into one correlator. Each insertion is placed in a distinct index namespace before Wick contraction. The corresponding position labels in the momentum map are renamed at the same time. This prevents a source index and a sink index from becoming accidentally identified merely because the two independently constructed operators happened to use the same dummy label. Intentional contractions inside each operator remain unchanged.

For a two-point correlation matrix, the sink and source bases are retained as separate ordered objects. When an explicit source basis is not supplied, the source operators are constructed as the Hermitian conjugates of the sink operators; otherwise the two bases may be specified independently. This keeps source–sink orientation, flavor content, momentum, and open-index structure explicit before the Wick-contraction stage determines the allowed propagation lines.

3.2. Constructing composite operators

Composite operators are constructed by multiplying the elementary symbolic objects introduced above while preserving their order and typed-index connectivity. This gives one common representation for operator structures of different hadronic complexity rather than requiring a separate symbolic language for each hadron family. Schematically, a local meson-like operator may have the form

$$\mathcal{O}_M(x) = \bar{q}_{f_1}(x)_{a\alpha}\Gamma_{\alpha\beta}q_{f_2}(x)_{a\beta}, \tag{8}$$

whereas a local three-quark structure can be written as

$$\mathcal{O}_B(x)_\delta = \epsilon_{abc}(\Gamma_D)_{\alpha\beta} q_{f_1}(x)_{a\alpha} q_{f_2}(x)_{b\beta} q_{f_3}(x)_{c\delta}. \quad (9)$$

These equations illustrate the frontend composition rules rather than imposing one fixed physical choice of Γ, Γ_D, or flavor content. SymCorr provides a convenience constructor for the second pattern, but the result is the same ordinary symbolic operator representation used elsewhere in the frontend.

Spatially extended operators are built by attaching transformations to the relevant constituent fields before forming the product. A derivative, parallel-transport path, shift, or projection therefore remains associated with the field on which it acts. Likewise, explicit gauge links and color tensors can appear as factors in the expression when the operator definition requires them. This allows local and spatially structured operators to share the same algebraic composition rules.

Products can also combine complete operators. This is the mechanism used to retain constituent structure in multi-local or multi-hadron constructions. If H_1 and H_2 are independently defined operators with distinct locality labels and momentum assignments, their product has the schematic form

$$\mathcal{O}_{12} = H_1(\mathbf{p}_1)\, H_2(\mathbf{p}_2), \qquad \mathbf{P} = \mathbf{p}_1 + \mathbf{p}_2. \quad (10)$$

The symbolic expressions are concatenated and the constituent momentum maps are merged. Position labels associated with momentum-bearing constituents must be distinct when such operators are combined; an accidental collision is rejected rather than silently identifying two locality groups. In this way the constituent momenta and the total momentum remain simultaneously available to later stages.

A second form of composition is a formal linear combination of complete basis operators,

$$\mathcal{O} = \sum_i c_i\, \mathcal{O}_i. \quad (11)$$

This is represented separately from the internal product structure of each $\mathcal{O}_i$. The frontend supports addition, subtraction, scalar multiplication, and products of such combinations. Multiplication is distributive, so the product of two sums generates the corresponding products of basis operators. Equivalent basis terms are combined by their coefficients, and terms whose resulting coefficient is numerically zero at the frontend tolerance are removed.

Hermitian conjugation acts on the full symbolic construction rather than only on its outer label. For a product, the factor order is reversed and each factor is conjugated according to its symbolic rule; for a linear combination the coefficients are complex conjugated and each basis operator is daggered. The associated momenta reverse sign as discussed above. These operations allow a source operator to be derived from a sink definition without manually reconstructing its field ordering, spin structure, or momentum orientation.

Correlators can contain either basis operators or such linear combinations. The latter are not passed to Wick contraction as an opaque sum. Instead, the correlator is expanded into weighted basis-operator correlators, after which Wick contraction is applied to each definite symbolic field product. This keeps linearity separate from the fermionic pairing problem and ensures that the coefficients multiplying the final contraction terms remain explicit.

The practical consequence is that meson-like, baryon-like, tetraquark, and multi-hadron constructions can be assembled from the same small symbolic vocabulary. Their differences reside in the field content, tensor connectivity, transformations, momenta, and linear combinations specified by the operator, not in separate downstream execution interfaces.

3.3. Simple operator examples

The following two examples illustrate how the mathematical expressions above are represented in the current frontend. They are intended only to demonstrate operator construction; the external-data validation status of particular channels is discussed separately in Sec. 7.

As a first example, consider the zero-momentum pseudoscalar operator

$$\mathcal{O}_{\mathrm{PS}}(x) = \bar{d}(x)\, \gamma_5\, u(x). \quad (12)$$

Using explicit color and spin indices, the corresponding SymCorr definition is

```
pion = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * u("x", "a", "beta"),
    momentum=(0, 0, 0),
)
```

Here the repeated color label `a` forms the color singlet, while the repeated spin labels connect the quark fields through γ_5. The position label `x` defines one locality group and therefore allows the single momentum assignment to be attached unambiguously. A corresponding source operator can be produced from the same definition,

```
source = pion.dagger()
corr = Correlator(pion, source)
```

without inserting a quark propagator or writing the Wick-contracted expression by hand. Those objects are generated only in the subsequent compilation stages.

For a three-quark example, the repository provides a convenience constructor for a local baryon-like operator of the form

$$\mathcal{O}_B(x)_\delta = \epsilon_{abc}(C\gamma_5)_{\alpha\beta} \ell(x)_{a\alpha} \ell(x)_{b\beta} s(x)_{c\delta}, \quad (13)$$

where ℓ denotes the generic light-flavor field available in the frontend. The representative compiler fixture constructs this operator as

```
baryon = build_baryon_operator(
    ["l", "l", "s"],
    "Cg5",
    momentum=(0, 0, 0),
)

corr = CorrMatrix([baryon])[0, 0]

proton = Operator(
    epsilon("a", "b", "c")
    * cgamma5("alpha", "beta")
    * u("x", "a", "alpha")
    * d("x", "b", "beta")
    * u("x", "c", "delta"),
    momentum=(0, 0, 0),
)

    tetraquark = Operator(
    epsilon("a", "b", "e")
    * epsilon("c", "d", "e")
    * u("x", "a", "alpha")
    * cgamma5("alpha", "beta")
    * d("x", "b", "beta")
    * ubar("x", "c", "gamma")
    * cgamma5("gamma", "delta")
    * dbar("x", "d", "delta"),
    momentum=(0, 0, 0),
)

    pion_dx = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * derivative(
        u("x", "a", "beta"),
        "+x",
    ),
    momentum=(0, 0, 0),
)

    pion_tx = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * transport(
        u("x", "a", "beta"),
        "+x",
    ),
    momentum=(0, 0, 0),
)

    pion_txy = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * transport(
        u("x", "a", "beta"),
        "+x",
        "+y",
    ),
    momentum=(0, 0, 0),
)

    two_meson_local = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * u("x", "a", "beta")

    * ubar("x", "b", "gamma")
    * gamma5("gamma", "delta")
    * d("x", "b", "delta"),

    momentum=(0, 0, 0),
)

    meson1 = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * u("x", "a", "beta"),
    momentum=(1, 0, 0),
)

meson2 = Operator(
    ubar("y", "b", "gamma")
    * gamma5("gamma", "delta")
    * d("y", "b", "delta"),
    momentum=(-1, 0, 0),
)
two_meson_bilocal = meson1 * meson2

    two_meson_bilocal = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * u("x", "a", "beta")

    * ubar("y", "b", "gamma")
    * gamma5("gamma", "delta")
    * d("y", "b", "delta"),

    momenta={
        "x": (1, 0, 0),
        "y": (-1, 0, 0),
    },
)
```

The helper inserts the antisymmetric color tensor and the selected diquark spin matrix but returns the same ordinary symbolic `Operator` used by the rest of the frontend. In the one-element correlation matrix shown above, no explicit source basis is supplied, so SymCorr constructs the source by Hermitian conjugating the sink operator before the correlator is formed.

The contrast between these two examples is useful for the remainder of the paper. Increasing the number of quark fields changes the possible Wick pairings and the eventual contraction topology, but it does not require a new frontend

representation. Both operators enter the compiler as ordered symbolic field expressions with explicit flavor, index connectivity, spin/color structure, and momentum information. The automatic Wick-contraction stage is therefore the first point at which their substantially different propagation structures emerge.

Derivative and displaced operators are written by applying `derivative`, `transport` or `shift` to the field on which the operation acts. Fig. 2 shows the three constructions used in the validation of Sec. 7, together with the operators they define. The lattice operations follow the conventions

$$\begin{aligned} T_{+i}\psi(x) &= U_i(x)\,\psi(x+\hat{\imath}), \\ T_{-i}\psi(x) &= U_i^{\dagger}(x-\hat{\imath})\,\psi(x-\hat{\imath}), \\ \nabla_i &= \tfrac{1}{2}\left(T_{+i}-T_{-i}\right), \end{aligned} \tag{14}$$

with periodic spatial boundaries and no lattice-spacing factor, where U_i are the stout-smeared spatial links of the gauge configuration that also define the Laplacian eigenvectors. A path is listed from the field position towards the displaced endpoint, so that $T_{(+x,+y)}\psi(x) = U_x(x)\,U_y(x+\hat{x})\,\psi(x+\hat{x}+\hat{y})$. The source operators are obtained with `dagger()`, which keeps the operation chain on the conjugated field.

4. Automatic Wick Contraction

4.1. Flavor-compatible contractions

Wick contraction begins only after any operator linear combinations have been expanded into definite basis correlators. For one such correlator, SymCorr collects the quark and antiquark fields from all operator insertions while retaining their original left-to-right ordering. The first combinatorial task is then to determine every complete pairing allowed by the explicit quark flavors carried by those fields.

For each flavor f, let Q_f denote the set of quark fields and $\bar{Q}_f$ the set of antiquark fields appearing in the correlator. A complete Wick contraction requires

$$|Q_f| = |\bar{Q}_f| \qquad \text{for every flavor } f. \tag{15}$$

If this condition fails for any flavor, the current Wick engine produces no complete contraction term for that basis correlator. Cross-flavor pairings are never introduced to compensate for the mismatch.

When the flavor counts are balanced, every bijection between the quark and antiquark fields of the same flavor is generated. If a flavor occurs n_f times as a quark and n_f times as an antiquark, that flavor channel therefore contributes $n_f!$ possible pairings. Since the choices for different flavors are independent, the complete set of flavor-compatible pairings is the Cartesian product of the channel-wise choices, with total number

$$N_{\text{pair}} = \prod_f n_f!. \tag{16}$$

This enumeration is important once identical flavors occur several times in a multi-quark or multi-hadron operator: direct and exchange pairings arise from the same rule rather than being encoded as separate hand-written topologies.

Flavor matching is based on the symbolic QCD flavor, not on the identity of a numerical propagator resource that may eventually be used at run time. In particular, u and d remain distinct Wick flavors even if a numerical setup uses the same light-quark parameters or the same underlying data source for both. Isospin combinations or other flavor superpositions must therefore be specified explicitly through the operator linear-combination layer; the Wick engine does not infer them from numerical degeneracies.

The pairing search is also not restricted to fields belonging to different operator insertions. A quark and antiquark of the same flavor may be paired even when both occur on the same side of a two-point function or within the same composite insertion. Retaining these possibilities is necessary for closed quark loops and disconnected or mixed contraction topologies. Filtering the pairings to sink–source lines alone would silently remove such terms before their topology could be identified.

At this stage only the admissible field pairings have been determined. Each complete pairing is retained explicitly and is passed to the subsequent Wick steps, which determine its fermionic sign and replace each matched quark–antiquark pair by the corresponding symbolic propagation line. No numerical lattice data, contraction ordering, or hardware information enters this flavor-compatibility decision.

4.2. Fermionic signs

The relative sign of each Wick term is fixed by the ordering of the fermionic fields in the original symbolic correlator. SymCorr therefore retains the encounter order of all quark-field leaves across the ordered operator insertions. Spin matrices, color tensors, and spatial transformations remain part of the operator expression, but they do not themselves contribute to the Grassmann permutation parity.

Let the ordered fermionic fields in a basis correlator be $F = (f_1, \ldots, f_{2N})$, and consider one complete flavor-compatible pairing

$$\mathcal{P} = \left\{(q_{i_1}, \bar{q}_{j_1}), \ldots, (q_{i_N}, \bar{q}_{j_N})\right\}. \tag{17}$$

For the purpose of determining the sign, SymCorr forms the sequence in which each quark is placed immediately before its paired antiquark,

$$F_{\mathcal{P}} = (q_{i_1}, \bar{q}_{j_1}, \ldots, q_{i_N}, \bar{q}_{j_N}), \tag{18}$$

and determines the parity of the permutation that maps the original field ordering F to this paired ordering. If $I(\mathcal{P})$ is the number of inversions in that permutation, the Wick sign stored by the frontend is

$$s_{\mathcal{P}} = (-1)^{I(\mathcal{P})}. \tag{19}$$

This is equivalent to counting the parity of the adjacent exchanges of fermionic fields required to bring every contracted pair together. The order in which complete pairs are listed does not affect the result: exchanging two whole pairs moves two fermionic fields past two others and therefore changes the permutation by an even number of transpositions.

A four-field example makes the rule explicit. Suppose the original fermionic order is

$$q_1\, q_2\, \bar{q}_1\, \bar{q}_2. \tag{20}$$

For the pairing $(q_1, \bar{q}_1)(q_2, \bar{q}_2)$, the paired sequence corresponds to the position permutation $(1, 3, 2, 4)$, which contains one inversion and hence has sign -1. For the exchanged pairing $(q_1, \bar{q}_2)(q_2, \bar{q}_1)$, the position permutation is $(1, 4, 2, 3)$, which contains two inversions and hence has sign $+1$. Thus the relative direct/exchange sign is obtained from the same general permutation rule used for larger multiquark systems; it is not inserted as a special topology-dependent convention.

The sign and the algebraic coefficient are kept as separate pieces of the Wick term. For a basis correlator the contraction engine initially associates unit coefficient with each generated pairing and stores its Grassmann parity separately. If the correlator originated from a linear combination of operators, the corresponding complex weight multiplies the algebraic coefficient without altering the sign calculation. At the subsequent physics-representation boundary, these factors are combined into the coefficient multiplying the logical correlator,

$$c_{\mathcal{P}}^{\text{phys}} = s_{\mathcal{P}}\, c_{\mathcal{P}}. \tag{21}$$

Keeping the two quantities distinct during the Wick stage makes the origin of a minus sign inspectable before later canonicalization, contraction planning, or numerical execution. In particular, the numerical backend does not infer or repair fermionic signs from propagator orientation or tensor layout; the sign is already fixed by the symbolic field ordering.

4.3. Canonicalization and symbolic simplification

Canonicalization in SymCorr is used to identify symbolic descriptions that are mathematically equivalent under rules that are explicit in the implementation. It is not used to guess additional field-theory identities. In particular, the frontend retains the expression exactly as supplied by the user, while a separate structural identity removes dependence on arbitrary dummy-index names. Contracted (dummy) spin, color, and position labels are renamed deterministically, and the momentum map is transformed through the same position-label mapping. Open indices keep their names, so that, for example, an operator and its transpose in two open spin indices remain distinct. Ordered factor structure, flavor, transformations, and momentum values remain part of the operator identity, whereas user metadata is treated as annotation rather than symbolic physics.

A second canonicalization step occurs at the compiler boundary before Wick contraction. Field-operation chains are normalized without mutating the frontend operator. For pure coordinate shifts and gauge-covariant parallel transport, adjacent inverse lattice hops cancel. For example,

$$(+x, -x, +y) \longrightarrow (+y). \tag{22}$$

Nested field operations are then fused into a compiler-internal composite operation that preserves their order. Other primitive operations are retained unchanged by this pass, and the operator momentum assignments and metadata are copied to the normalized compiler-side representation.

This normalization is deliberately performed before linear-combination terms are merged. Two frontend operators can therefore remain visibly different as user objects yet become the same compiler basis operator when, for example, their transport paths differ only by adjacent inverse hops. Once normalized, weights multiplying identical ordered basis-operator tuples are added, and a basis term is removed only when its weight cancels exactly; no numerical tolerance is applied. For a two-point correlation matrix, the corresponding simplification pass performs the same coefficient aggregation separately within each matrix element for identical normalized sink–source basis pairs.

The same normalized operator identity is also used at the Wick-cache boundary. The cache key is the complete ordered tuple of basis-operator insertions rather than an unordered set. Consequently, repeated occurrences of the same normalized symbolic correlator can reuse an already generated Wick expansion, while a physically different operator ordering remains a distinct contraction problem. This reuse avoids repeated symbolic work but does not alter the set of Wick pairings or their fermionic signs.

The scope of these transformations is intentionally restricted. The pre-Wick normalizer does not commute or reorder fermion factors, replace spin or color structures using additional identities, infer cyclic trace equivalences, or simplify expressions using numerical tensor values. Such transformations could change the field ordering or require assumptions that belong to later compiler representations. At this stage SymCorr therefore applies only the canonicalization rules for which equivalence is explicit in the symbolic algebra.

There is a separate simplification mechanism later in the compilation pipeline, after the Wick terms have been lowered to numerical contraction graphs. At that stage exact global common-subexpression analysis can canonicalize reusable tensor-network components and combine exactly equal complete Wick terms within a correlator. Like the pre-Wick weights, the later complete-term aggregation removes only exactly cancelling coefficients, and it does so only for scalar terms. Keeping these two stages distinct avoids conflating symbolic algebra with numerical contraction reuse; the latter is discussed in Sec. 6.

4.4. Output of the contraction stage

For each normalized basis correlator, the Wick engine returns an ordered collection of symbolic Wick terms. A convenient schematic description of one such term is

$$W_\alpha = \left(s_\alpha, c_\alpha, \mathcal{P}_\alpha, \mathcal{E}_\alpha, \mathcal{C}_\alpha\right), \tag{23}$$

where s_α is the fermionic sign, c_α is the algebraic coefficient, $\mathcal{P}_\alpha$ is the explicit set of paired quark and antiquark fields, $\mathcal{E}_\alpha$ is the contracted symbolic expression, and $\mathcal{C}_\alpha$ records the locality-graph components inferred from the pairing. These quantities remain symbolic: no perambulator, distillation kernel, tensor shape, external file, memory buffer, or execution device has yet been introduced.

The pairing $\mathcal{P}_\alpha$ is retained directly rather than being recoverable only from a printed algebraic expression. Each entry identifies the quark endpoint and the antiquark endpoint of one logical propagation line, including their flavor and symbolic spin, color, and position labels. This explicit pairing is the authoritative input used by the next compiler boundary to construct logical quark lines. It also retains the information needed to associate field-level derivatives, transports, shifts, and spatial representations with the correct endpoints.

The symbolic expression $\mathcal{E}_\alpha$ retains the non-fermionic tensor content inherited from the original operators and represents contracted quark lines symbolically. In particular, spin matrices, color tensors, gauge-link objects, and their index connectivity remain visible rather than being replaced by numerical arrays at the Wick stage. The later physics-lowering pass reads these symbolic tensors and the explicit pairing together; it does not infer the quark-line topology from numerical tensor shapes.

In addition to the algebraic term, SymCorr computes a locality graph from the pairing. Position labels form the graph vertices and a contracted quark line connects the position labels of its two endpoints. Each resulting graph component records both the locality labels that it contains and the operator insertions touched by those labels. The present Wick-level topology label is a coarse diagnostic: a term is classified as *disconnected* when all components are local to individual operator insertions, *connected* when there are spanning components and no purely local component, and *mixed* when spanning and local components occur together. This terminology does not assert that the complete numerical tensor network consists of one connected component.

For composite operators, the same component records provide an additional structural diagnostic. The current `factorized` property is true when no locality component spans more than two operator insertions; a component touching more than two insertions marks a multi-operator exchange structure. This information is useful for inspecting the generated Wick terms, but it is kept separate from later numerical common-subexpression analysis and contraction planning.

Table 1
Number of Wick terms generated by SymCorr for $C(t,t_0) = \langle \mathcal{O}(t)\,\mathcal{O}^\dagger(t_0)\rangle$, separated into terms whose quark lines all connect source and sink (src–snk), connected terms containing equal-time lines, and disconnected terms with closed loops. The operators are $\pi^+ = \bar{d}\gamma_5 u$, $\bar{u}u$, $p = \epsilon_{abc}(u_a^\top C\gamma_5 d_b)u_c$ and $\Delta^{++} = \epsilon_{abc}(u_a^\top C\gamma_i u_b)u_c$; two-meson and two-baryon operators are products of separately projected hadrons, and $p\pi^+$ is local.

$\mathcal{O}$	terms	src–snk	equal-time	loops
π^+	1	1	0	0
$\bar{u}u$	2	1	0	1
p	2	2	0	0
Δ^{++}	6	6	0	0
$\pi^+\pi^+$	4	4	0	0
$\pi^+\pi^-$	4	1	2	1
$p\pi^+$	12	6	6	0
pp $(I{=}1,\ {}^1S_0)$	48	48	0	0

When a basis correlator originates from a linear combination, its previously computed complex basis weight is applied to the Wick-term coefficient while the cached sign, pairing, expression, and component structure remain unchanged. The next lowering step then combines sign and coefficient and translates each Wick term into a backend-independent logical correlator. In the current PhysicsIR, the resulting records contain logical quark lines, spin tensors, color tensors, gauge tensors, field-operation information, and momentum assignments. The Wick-level locality-component and topology diagnostics are not part of this PhysicsIR data contract.

This boundary is important because it marks the end of the purely symbolic Wick problem. By the time a term leaves this stage, SymCorr has fixed which fermion fields contract, the associated Grassmann sign, the algebraic weight, and the symbolic operator structures that accompany those propagation lines. The next stage may reorganize this information into backend-independent physics records, but it is not permitted to invent a new pairing or reinterpret the fermionic sign.

4.5. Contraction topologies

Because the pairing search is complete within each flavor and is not restricted to source–sink lines, every quark-flow topology allowed by the flavor content of the operators is generated automatically; no diagram is entered by hand or selected by the user. This includes quark lines that start and end on the same time slice and closed quark loops. Representative diagrams are shown in Fig. 3, and Table 1 lists the number of Wick terms that SymCorr generates for representative operators, separated into terms whose quark lines all connect source and sink, terms that contain equal-time lines, and disconnected terms with closed loops. The relative signs and the subsequent tensor-network construction follow from the same rules for all of them.

5. Lowering to the Distillation Representation

5.1. Distillation primitives

The input to distillation lowering is the validated PhysicsIR described at the end of Sec. 4. At this boundary the fermionic pairing and coefficient of each Wick contribution are already fixed. The purpose of the distillation layer is therefore not to redo the field-theory algebra, but to translate the physical records into a small set of objects that describe the spatial/color structures and propagation resources required by a distillation calculation.

The first primitive is a *projected field endpoint*. Every endpoint in PhysicsIR carries an explicit spatial representation, currently either LapH or POINT, together with any field-local derivative, transport, or shift operations. For a LapH endpoint, the compiler realizes the contribution according to the orientation of the fermion field,

$$q_{\mathrm{LAPH}} \longmapsto V, \qquad \bar{q}_{\mathrm{LAPH}} \longmapsto V^{\dagger}, \tag{24}$$

whereas an explicitly POINT endpoint remains a distinct point contribution. This translation records which spatial object is required; it does not load an eigenvector tensor or construct a numerical array. The operations attached to the original endpoint remain associated with the realized field so that their placement is not reconstructed later from tensor shape or ordering.

Before these projected fields are assembled, SymCorr partitions each logical correlator into spatial–color components. The partition is determined from the incidence of field endpoints, position labels, color labels, explicit color tensors, and gauge-link tensors. Each connected component containing at least one field endpoint becomes an operator vertex. Spin tensors do not determine this partition. Consequently, the decomposition is independent of whether the original symbolic object was described as a meson, baryon, tetraquark, or multi-hadron operator; it follows only from the explicit spatial and color connectivity already present in PhysicsIR.

A realized spatial–color component is represented by a `KernelRecipe`. Conceptually, this recipe specifies the operator-dependent projected object that will ultimately be built from the required eigenvector or point contributions, field-local spatial transformations, color tensors, gauge tensors, position variables, and momentum projections. For LapH fields its external spatial legs are LapH-mode indices; POINT fields remain distinguishable and retain a color-like external spatial leg. The recipe is therefore a compiler description of the object to be materialized rather than the materialized tensor itself.

Spin structure is kept as a separate primitive. Gamma matrices and other spin tensors have already been converted into explicit `SpinTensorRecord` objects in PhysicsIR, and they are deliberately excluded from the identity of a spatial/color `KernelRecipe`. This separation mirrors the distillation factorization discussed in Sec. 2: the reusable spatial object and the spin structure that contracts with it need not be treated as one opaque tensor at the compiler boundary. It also allows two contributions with the same spatial/color construction but different spin structure to share the same spatial resource where the later contraction graph permits it.

The remaining primitive class describes quark propagation between these operator-dependent objects. From the exact logical quark lines, later stages reconstruct open fermion paths and closed cycles and assign corresponding propagation requirements. For an open path this requirement is represented by a `PropagationRecipe`; with no inserted current it denotes an ordinary perambulator requirement, while supported explicit POINT-current insertions can produce a generalized propagation requirement. The detailed construction and current implementation boundary of these resources are discussed below in the subsection on propagation resources.

These objects should be distinguished from concrete numerical data. At this stage a kernel recipe is not yet an evaluated elemental, and a propagation recipe is not an in-memory perambulator. No filename, HDF5 group, source time, tensor shape, physical buffer, CPU/GPU placement, or contraction schedule is chosen by distillation realization. Those choices belong to later graph, resource-binding, materialization, and execution stages. The output of this layer is therefore a more concrete statement of the required distillation calculation while remaining independent of a particular data layout or machine.

5.2. Operator-dependent structures

The operator-dependent part of the distillation calculation is constructed from the spatial–color components identified above rather than from a fixed library of meson, baryon, or tetraquark tensor shapes. For one component A, let $\mathbf{x}_r$ denote its position variables, $\mathbf{p}_r$ the associated momenta, and k_i the external LapH-mode indices carried by its projected fields. A useful schematic form of the corresponding spatial kernel is

$$\mathcal{K}_A(\{k_i\};t) = \sum_{\{\mathbf{x}_r\},\{a_i\}} \exp\!\left(i \sum_r \mathbf{p}_r\cdot\mathbf{x}_r\right) C_A(\{a_i\}) \times \mathcal{U}_A(\{\mathbf{x}_r\},\{a_i\}) \prod_i \mathcal{V}_i(\mathbf{x}_{r(i)}, a_i, k_i; t), \tag{25}$$

where C_A denotes the explicit color-tensor structure, $\mathcal{U}_A$ denotes any explicit gauge-link structure represented in the component, and $\mathcal{V}_i$ denotes the appropriately oriented LapH factor, including the field-local spatial operations attached to that endpoint. For a quark endpoint this factor is built from V, while for an antiquark endpoint it is built from the conjugate/projected orientation corresponding to $V^{\dagger}$. Equation (25) is schematic: its purpose is to display the information retained by the compiler rather than to impose one fixed tensor rank or one fixed pattern of spatial indices.

For example, a local bilinear whose two LapH fields are contracted to a color singlet gives a two-mode spatial kernel of the schematic form

$$K^{(\mathbf{p})}_{kl}(t) = \sum_{\mathbf{x},a} e^{+i\mathbf{p}\cdot\mathbf{x}} \left[\mathcal{D}_{\bar{q}} V(t)\right]^{*}_{\mathbf{x}ak} \left[\mathcal{D}_{q} V(t)\right]_{\mathbf{x}al}, \quad (26)$$

where $\mathcal{D}_q$ and $\mathcal{D}_{\bar{q}}$ stand for the spatial operations attached to the two field endpoints. The spin matrix is not folded into this kernel. In the simplest separable case the familiar elemental-like object is obtained only after the independent spin structure is attached, for example through a factor $\Gamma_{\alpha\beta} K_{kl}$. This is the concrete reason that SymCorr does not use the full spin–spatial elemental as its reusable spatial resource identity.

The same construction applies without changing the lowering algorithm when the component has higher rank. For a local three-quark color-singlet structure, the spatial part has the schematic form

$$B^{(\mathbf{p})}_{k_1k_2k_3}(t) = \sum_{\mathbf{x},a,b,c} e^{+i\mathbf{p}\cdot\mathbf{x}} \epsilon_{abc} \left[\mathcal{D}_1 V(t)\right]_{\mathbf{x}ak_1} \times \left[\mathcal{D}_2 V(t)\right]_{\mathbf{x}bk_2} \left[\mathcal{D}_3 V(t)\right]_{\mathbf{x}ck_3}. \quad (27)$$

The diquark spin matrix and any remaining open spin index are represented separately. Thus the difference between the bilinear and three-quark examples appears through the number of projected fields and their explicit color connectivity, not through a hadron-specific branch in the distillation compiler.

Momentum is part of the spatial resource itself. SymCorr retains one momentum assignment for each position group in a component and uses the phase convention $e^{+i\mathbf{p}\cdot\mathbf{x}}$; Hermitian conjugation has already reversed the corresponding source momentum where required. For a component containing several independent position labels, the spatial object contains the product of those phase factors, equivalently the exponential shown in Eq. (25). This keeps constituent momenta visible for bilocal and multi-local constructions rather than replacing them by only the total momentum.

The compiler assigns each `KernelRecipe` a deterministic spatial resource identity. That identity is invariant under dummy position/color renaming but includes the projected-field orientations, field-local operations, color tensors, explicit gauge connectivity, and momentum assignments. Spin and quark flavor are deliberately excluded from this spatial identity. Spin appears as separate primitive tensor nodes in the subsequent contraction graph, while the LapH eigenvector subspace itself is a gauge-field spatial object rather than a flavor-dependent quark propagator. Equal spatial resource identities therefore expose possible reuse without yet merging the distinct occurrences that appear in different Wick terms or matrix elements; that global reuse is a later optimization step.

For the currently executable exact-distillation path, the compatibility bridge turns a spatial recipe into primitive operations for the required eigenvector factors, momentum phases, and the supported δ and ϵ color tensors. Derivative information remains attached to the field and can be realized by the numerical tensor provider from derivative eigenvector data or, when configured, from the underlying eigenvectors and gauge links. The recipe layer is broader than this executable subset: explicit `GaugeLink` tensor nodes can be represented in the spatial/color component, but the current numerical kernel ISA rejects such standalone gauge-tensor nodes rather than silently approximating them. This distinction is retained here so that symbolic representability is not presented as equivalent to validated numerical execution.

The result of this lowering is therefore an operator-dependent spatial resource with explicit LapH ports, together with separate spin tensors whose indices are connected only when the primitive contraction graph is built. This is the form in which SymCorr carries the operator structure forward to the propagation and contraction stages: sufficiently concrete to determine the required numerical tensors, but still independent of the contraction order, memory layout, or execution device.

5.3. Propagation resources

After the operator-dependent kernels have been identified, the Wick quark lines must be mapped to the propagation tensors that connect their spatial and spin ports. SymCorr performs this step from the exact fermion topology rather than from operator names or tensor shapes. Each open quark path is decomposed into its ordered Wick-propagator segments and any operator-local transitions that join successive segments. The result is a logical propagation requirement; the compiler still does not load or construct a numerical propagator.

For an ordinary path containing one Wick segment and no absorbed insertion, the required object is the usual distillation perambulator introduced in Sec. 2. The corresponding `PropagationRecipe` records the exact source and sink spatial kernels, the particular open port on each kernel, the flavor carried by the Wick segment, and the spatial representation of the two endpoints. Schematically, its numerical realization has the canonical index structure

$$\tau_f(t_{\mathrm{snk}}, t_{\mathrm{src}}) \equiv \tau_{f;\,k\alpha,\,l\beta}(t_{\mathrm{snk}}, t_{\mathrm{src}}), \quad (28)$$

where (k, α) are the sink LapH-mode and spin indices and (l, β) are the source indices. The orientation is inherited from the Wick pairing and is therefore fixed before any external storage convention is considered.

Open paths can also contain explicit local current insertions. In the current compiler, a current is absorbed into a generalized propagation resource only when its quark and antiquark endpoints are explicitly represented as POINT fields belonging to one spatial–color component. For one insertion, the logical structure is schematically

$$\mathcal{T}^{J}_{f_0\to f_1}(t_2, t_1, t_0), \quad (29)$$

where the propagation segment before the current carries flavor f_0, the segment after it carries flavor f_1, and J

denotes the specified local current structure. SymCorr does not assume that the two flavors are equal. A flavor-changing current is represented as a change between adjacent propagation segments rather than as a flavor change inside one Wick segment.

The generalized recipe retains substantially more information than only the name of the current. For every absorbed insertion it stores the spatial kernel, the exact incoming and outgoing kernel ports, the incoming and outgoing flavors, the endpoint spin labels, and the independent spin tensors connecting those labels. Several POINT bilinear insertions can be retained in path order by the compiler representation. The deterministic propagation-resource identity therefore depends on whether the object is ordinary or generalized, the source and sink representations, the ordered sequence of segment flavors, and the canonical spatial and spin content of the insertions. It deliberately does not depend on a filename, HDF5 dataset name, source time, device, or execution strategy.

This logical identity is separated from the occurrence at which the resource is used. During resource binding, each compiler propagation resource is mapped to one exact external tensor source. The occurrence placement then supplies the frontend source, insertion, and sink operator slots, and the run-time operator-time context converts those slots into concrete Euclidean times. Thus two uses of the same mathematical propagation resource can request different time slices without becoming different compiler resources. Conversely, the binding layer does not use a fallback such as "look up a tensor by flavor"; ordinary and generalized resources are bound by their exact resource identity.

External storage conventions are normalized only when a concrete propagation tensor is materialized. The canonical tensor expected by the contraction graph is ordered as sink mode/spin followed by source mode/spin. For a fixed-source file containing $P_{\mathrm{file}}(t, t_0)$, a direct request with $t_{\mathrm{src}} = t_0$ uses the stored orientation after the configured axis conversion. If instead $t_{\mathrm{snk}} = t_0$, SymCorr reconstructs the reverse propagation using γ_5-Hermiticity,

$$P(t_0, t) = \gamma_5 \, P_{\mathrm{file}}(t, t_0)^{\dagger} \, \gamma_5. \tag{30}$$

The Hermitian adjoint exchanges both mode and spin source/sink spaces; a simple complex conjugation is not equivalent. If neither requested endpoint is the fixed source, that tensor cannot be obtained from the file and materialization fails rather than inventing missing propagation data.

The materialization adapter also owns external source-field and spin-basis conventions. In the validated QUDA-LapH input path, for example, a stored `qdagger` source convention is converted to SymCorr's canonical $\langle q\bar{q} \rangle$ convention using the configured spin basis before the tensor reaches the generic numerical executor. Likewise, a reverse fixed-source transformation is applied in the adapter exactly once. The contraction executor therefore consumes the propagation tensor in the orientation requested by the compiler and does not apply an additional post-hoc conjugation or γ_5 correction.

Closed fermion cycles use a related but distinct lowering. For an exact LapH closed cycle, SymCorr does not request a separately stored disconnected-loop tensor. Instead, every Wick segment is represented by an ordinary perambulator and the successive operator-local spatial and spin tensors close the cyclic contraction. This route currently assumes ordinary all-time distillation perambulators for the required segments. A genuinely POINT disconnected loop cannot be reconstructed from those perambulators because they do not contain the full point-space all-to-all propagator; such a loop therefore requires additional stochastic, all-to-all, or precomputed-loop information and is rejected by the present exact-distillation lowering.

The compiler and resource-binding layers can consequently represent ordinary perambulators, ordered generalized POINT-current resources, and exact-LapH closed-cycle propagation structure within one topology-driven framework. Their validation status is nevertheless different. The ordinary perambulator path is part of the external-data validation described in Sec. 7, whereas generalized propagation inputs and the broader closed-loop pathways should be regarded as implemented compiler/runtime capabilities unless a separate external numerical validation is stated later in this paper.

5.4. Construction of contraction tasks

Once the operator-dependent kernels, propagation requirements, and spin tensors have been identified, SymCorr converts each complete PhysicsIR Wick contribution into a primitive tensor-contraction graph. This graph is the first compiler representation in which the numerical contraction problem is explicit. It contains the tensors that must participate in the Wick term and the indices that connect them, but it still contains no decision about the order in which those tensors will be contracted.

For a Wick term α, let $T_{\alpha n}$ denote its primitive tensor occurrences and let $I_{\alpha n}$ denote the ordered set of contraction indices attached to tensor n. The graph represents the contribution schematically as

$$C_\alpha = c_\alpha \sum_{\{i\}_{\mathrm{internal}}} \prod_n T_{\alpha n}(I_{\alpha n}), \tag{31}$$

where c_α is the coefficient already fixed in PhysicsIR. Indices that occur on two primitive tensor legs are summed internal indices. An index that occurs on only one leg remains a free output index, so the same graph representation can describe both scalar correlator terms and terms with an explicit open index. The graph validator rejects an index attached to more than two primitive legs rather than assigning an ambiguous multiway contraction meaning.

The current primitive graph has only three tensor kinds: operator-dependent spatial kernels, propagation tensors, and spin tensors. A kernel node exposes one port for every surviving external spatial leg of its `KernelRecipe`. For a

LapH kernel these ports become LapH-mode contraction indices. A spin node exposes its left and right spin indices. A propagation node connects the corresponding kernel and spin spaces using the canonical leg order

$$(\text{sink mode}, \text{sink spin}, \text{source mode}, \text{source spin}). \quad (32)$$

The graph therefore records tensor connectivity explicitly rather than relying on a later numerical backend to infer contractions from array shape or node ordering.

Fermion topology determines which propagation nodes are inserted. Along an open fermion path, POINT bilinear transitions that satisfy the generalized-resource requirements described above are absorbed into the corresponding propagation recipe. Their spatial kernel and spin tensors are then omitted as separate graph nodes so that the current is not counted twice. A non-absorbed transition must currently have LapH endpoints and remains explicit through its kernel and spin tensors. Similarly, a standalone POINT spatial kernel is not admitted to the current exact-distillation primitive graph; connected POINT bilinears must be realized through the supported generalized propagation route. Mixed POINT/LapH transitions that cannot be represented by these existing resources are rejected at this boundary rather than assigned a guessed tensor realization.

Each primitive tensor occurrence carries two different identities. The *node identity* distinguishes the occurrence inside this particular Wick term, while the *resource identity* describes the reusable mathematical tensor that the occurrence requires. Two kernel occurrences, for example, may have the same spatial resource identity but different node identities because they occupy different places in the term. The primitive-graph stage intentionally preserves both occurrences. No deduplication is performed merely because their resource identifiers agree.

This distinction is necessary because resource equality and tensor occurrence are different statements. A Wick term may use the same perambulator resource more than once, and those two uses must retain distinct contractions even though they may ultimately be materialized from the same external data source. The same applies to repeated spatial kernels and spin matrices. Occurrence-specific ports and index bindings therefore remain attached to each node until a later global analysis proves what can be reused without changing the contraction.

The graph is validated before it is passed to optimization. Node identifiers must be unique, a tensor cannot expose the same local port twice, contraction indices cannot be empty, and no global index may have degree greater than two. A graph in which every index has degree two is a closed scalar contraction; degree-one indices define its external interface. These checks establish the well-formed numerical tensor network independently of any proposed contraction path.

SymCorr constructs one such `WickContractionGraph` for every complete PhysicsIR Wick term. Equal primitive resources therefore remain visible across terms and across correlation-matrix elements, but they have not yet been combined. The next stage operates on this collection globally: it deduplicates exactly equal resource requirements, canonicalizes reusable connected tensor components, and aggregates exactly equal complete terms before contraction-order selection. Those operations belong to the execution and optimization framework in Sec. 6; they are deliberately separated from the task construction described here.

At the end of this subsection the physics lowering is complete. The compiler has fixed the coefficient, the required primitive tensor resources, and their exact index connectivity. It has not chosen pairwise contraction order, common intermediate expressions, execution schedule, memory buffers, hardware device, or numerical tensor values. Those remaining decisions can therefore optimize an already fixed tensor problem without modifying the physical correlator.

6. Execution and Optimization Framework

6.1. Backend-independent execution representation

The execution layer begins from the collection of primitive Wick-contraction graphs constructed in Sec. 5. Its first purpose is to turn those term-local tensor networks into one hardware-independent representation of the complete requested calculation. The representation is progressively refined through global exact reuse, contraction-order selection, partial-intermediate reuse, and finally an explicit logical dependency graph. At every one of these stages the compiler is still describing mathematical values and operations rather than a schedule for a particular CPU or GPU.

The first global representation is a `GlobalComputationDAG`. Primitive resource requirements are identified by exact resource identity, while tensor occurrences retain their operator-slot placement. Connected pieces of the primitive tensor network are canonicalized independently of temporary node names and dummy contraction-index labels. One canonical component computation can therefore be referenced by several Wick terms or several correlation-matrix elements without treating occurrence-specific placement as part of the reusable resource itself. The requested correlators remain explicit as sums of Wick terms, so global reuse does not collapse distinct physical outputs.

Each unique connected component is then assigned a backend-independent contraction plan. SymCorr presently delegates the algebraic path search to `opt_einsum`, but stores the result in its own compiler representation. A plan is an ordered sequence of contraction or reduction steps; every step records its input values and interfaces, the indices eliminated at that step, the output interface, an estimated operation count, and the size of the resulting intermediate. This separates the mathematical contraction tree from the library used to discover it and leaves room for a different path optimizer without changing the surrounding compiler stages.

Contraction-path search requires numerical dimensions in order to estimate cost. In the current exact-distillation model, LapH-mode indices have dimension N_v for the job

and spin indices have dimension four. These dimensions are provided through a size model used for path selection,

$$|i| = \begin{cases} N_v, & i \in \text{LapH-mode space}, \\ 4, & i \in \text{Dirac-spin space}. \end{cases} \tag{33}$$

They are optimization inputs rather than part of tensor-resource identity. A change in N_v, or in the path-search strategy, can therefore change the chosen contraction order without changing the symbolic correlator or its primitive tensor connectivity.

Once a contraction tree has been selected, SymCorr rewrites its intermediate values into a second global DAG. Primitive tensor values are distinguished by both reusable resource identity and exact logical placement, while contraction subtrees are canonicalized from their recursively defined inputs and interfaces. Temporary component step names and dummy local indices are excluded from this identity. As a result, two algebraically identical partial contractions can refer to the same logical intermediate even when they arose in different component plans. Importantly, this pass does not alter the contraction order that was already selected; it only recognizes identical values inside those frozen trees.

The final compiler representation before scheduling is the `LogicalExecutionDAG`. It makes every remaining operation explicit. Its nodes fall into four classes:

$$\begin{aligned} v_r &= \text{EvalPrimitive}(r; \pi), \\ v_m &= \text{Contract}(v_{a_1}, \ldots, v_{a_k}), \\ w_\alpha &= c_\alpha \prod_j v_{\alpha j}, \\ C_i &= \sum_{\alpha \in i} w_\alpha, \end{aligned} \tag{34}$$

where r is a primitive resource, π is its logical operator placement, v_m is one reusable contraction intermediate, w_α is one Wick-term product, and C_i is one requested correlator output. Repeated uses of the same value remain distinct occurrences in an operation when multiplicity is part of the algebra, for example when the same disconnected scalar component appears twice in one Wick term.

The logical DAG therefore answers a precise question: *what values must be produced, and which earlier values does each operation depend on?* Primitive nodes have no dependencies; contraction nodes depend on their input values; Wick-term nodes multiply their component results by the already fixed coefficient; correlator-sum nodes depend on the contributing Wick terms. The output map identifies the final node corresponding to every requested correlator. Dependency cycles, missing values, incompatible value interfaces, and inconsistent Wick-term or correlator output indices are rejected by the logical-DAG validator.

A backend-neutral static analysis can subsequently derive exact logical tensor metadata from this DAG. Mode and spin index spaces remain explicit even when they happen to have the same numerical dimension, and each value receives its logical shape, element count, scalar type, byte estimate, producer/consumer relationships, fanout, and distance to an output. This analysis still describes one logical value at one exact placement; time batching, data residency, file layout, and physical memory slots are deliberately absent.

No representation described in this subsection is a physical execution plan. In particular, a topological ordering of the logical DAG merely demonstrates that its dependencies are acyclic; it is not the selected runtime schedule. Likewise, estimated FLOPs and intermediate sizes used during contraction-path search do not choose a device or allocate memory. Scheduling, lifetime-aware buffer reuse, time-context batching, staging, and hardware feasibility are introduced only by the later planning stages described below. This boundary allows those deployment choices to change while preserving the same optimized mathematical execution DAG.

6.2. Reuse of repeated structures

A central capability of the SymCorr framework is the identification and elimination of redundant computational work across an entire correlation matrix. Rather than treating each matrix element $C_{ij}(t - t_0)$ and each contributing Wick term as an independent contraction program, SymCorr formulates the global calculation as a unified computation directed acyclic graph (`GlobalComputationDAG`). This subsection explains how Wick-contracted terms are represented as attributed graphs, the multi-tier identifier hierarchy used throughout the compiler, the graph-isomorphism algorithm that discovers topologically equivalent contractions, and how this optimization operates in practical hadron correlation matrices.

6.2.1. Graph representation of hadron contractions: The proton example

To illustrate the graph formulation concretely, consider the rest-frame, positive-parity nucleon (proton) interpolating operator,

$$\mathcal{O}_N(t) = \epsilon_{abc} \left[u^a(t)^\top C \gamma_5 d^b(t) \right] P_+ u^c(t), \tag{35}$$

where $C = \gamma_2\gamma_4$ is the charge-conjugation matrix, ϵ_{abc} is the totally antisymmetric $SU(3)$ color tensor, and $P_+ = \frac{1}{2}(\mathbb{I}+\gamma_4)$ is the positive-parity energy projector. The corresponding Euclidean two-point correlation function is

$$C_{NN}(t, t_0) = \left\langle \mathcal{O}_N(t)\, \overline{\mathcal{O}}_N(t_0) \right\rangle, \tag{36}$$

where $\overline{\mathcal{O}}_N(t_0) = \mathcal{O}_N(t_0)^\dagger \gamma_4$ denotes the adjoint source operator.

Because the proton contains two valence up quarks of identical flavor, Wick's theorem generates two distinct contraction terms ($N_u! \times N_d! = 2! \times 1! = 2$):

1. **Direct term**: with fermionic permutation sign +1, where up quark 1 at the source propagates into up quark 1 inside the sink diquark, while up quark 2 propagates as the spectator quark.

2. **Exchange term**: with fermionic permutation sign -1, where the two up-quark lines cross: up quark 1 at the source propagates into the sink spectator, while up quark 2 propagates into the sink diquark.

In SymCorr, each Wick term is converted into an explicit tensor network graph (`WickContractionGraph`), displayed schematically in Fig. 4.

- **Nodes**: Each graph contains 12 primitive tensor nodes:
  - Two spatial baryon kernels (`kernel_0` at the sink and `kernel_1` at the source), each carrying three external LapH-mode ports: p_0, p_1, p_2.
  - Three quark perambulators ($\tau_u^{(1)}, \tau_d, \tau_u^{(2)}$), each carrying four ports: sink mode, sink spin, source mode, and source spin.
  - Seven Dirac spin tensors representing the source/sink $C\gamma_5$, P_+, and spinor conjugation structures.
- **Edges**: Edges correspond to contracted dummy indices (degree 2). A mode edge connects a perambulator mode port to a baryon kernel mode port. A spin edge connects a perambulator spin port to a Dirac matrix.

6.2.2. Identifier taxonomy in the compiler

To manage data reuse, compilation, and execution without name clashes or ambiguous sharing, SymCorr maintains four strictly decoupled tiers of identifiers:

1. **`correlator_id`**: Identifies the high-level physical correlator requested by the user in the job declaration (e.g., `c_proton` or `C_0_1`).
2. **`node_id`**: A diagram-local occurrence identifier assigned to a specific tensor node within one un-optimized Wick graph (e.g., `kernel_0`, `propagation_1`, `spin_3`).
3. **`resource_id`**: A globally unique identifier for the underlying physical/numerical data source. It is generated from a deterministic SHA-256 hash of the mathematical specification (e.g., `spatial_kernel_6427fc17...` and `spatial_kernel_63b068c2...` for the two proton baryon blocks, one built from V at the sink and one from $V^\dagger$ at the source, and `perambulator_cfa5757f...` and `perambulator_b3fea9be...` for the up- and down-quark perambulators; the values are those of the present version). The direct and exchange graphs have distinct node occurrences but share exactly these `resource_id` values.
4. **`computation_id`** (or **`component_id`**): Identifies a unique, canonicalized, connected tensor contraction network (e.g., `component_9b66414a...` for the direct graph and `component_301a5cf4...` for the exchange graph). This identifier is invariant under dummy index renaming and node reordering.

6.2.3. Graph isomorphism: Identifying identical contractions through canonical labeling

Two contraction graphs may represent the exact same mathematical contraction, yet appear entirely distinct to a computer. This occurs for two simple reasons:

1. **Dummy index naming**: Consider two mathematically identical contractions,

$$\sum_{\alpha} \tau_{\alpha\beta} (C\gamma_5)_{\beta\alpha} \quad \Longleftrightarrow \quad \sum_{\mu} (C\gamma_5)_{\nu\mu} \tau_{\mu\nu}.$$

   Although they compute the same tensor value, an automated parser sees different dummy labels (α, β versus μ, ν).
2. **Arbitrary node order**: Tensors in memory can appear in any sequence (listing τ before $C\gamma_5$ or vice versa) without altering the physical contraction result.

To recognize identical contractions across different correlators, SymCorr converts every contraction graph into a unique, standardized form—a process known as *canonical labeling*.

Redstar [5] performs this step with the general-purpose canonical-labeling package `nauty`. Lattice QCD contraction graphs are, however, compact, typically containing only 5–25 tensor nodes, so that an exhaustive search over starting nodes is affordable. SymCorr therefore uses a lightweight, self-contained canonicalization strategy based on three intuitive steps:

- **Step 1: Graph traversal and index standardization**: Starting from a chosen tensor, the compiler walks through the graph by following port connections in a fixed numerical port order (port 0, port 1, ...). As internal connections and open indices are encountered along the walk, they are renamed systematically to sequential canonical labels ($i_0, i_1, i_2, \ldots$ for contracted internal indices and $o_0, o_1, \ldots$ for open external indices).
- **Step 2: Unbiased root selection**: Because a loop of contracted tensors has no natural "first" node, SymCorr tries starting the walk from every candidate tensor in the graph. Each choice of starting node produces a standardized candidate string describing the walk.
- **Step 3: Deterministic tie-breaking (Lexicographical minimum)**: Among all candidate description strings generated across all possible starting nodes, the compiler deterministically selects the one that is alphabetically earliest (the lexicographical minimum).

Algorithm 1 Canonical Graph Isomorphism in SymCorr

Require: Contraction graph G with tensor nodes, port definitions, and index connections.
Ensure: Canonical identifier `computation_id` and standardized contraction plan.

1: **Initialize** candidate list $\mathcal{C} \leftarrow [\,]$.
2: **for** each candidate starting tensor r in G **do**
3: Walk through connected tensors starting from r, following port connections in fixed numerical port order.
4: Record the sequential visit order of tensors $(t_0, t_1, \ldots)$.
5: Encode tensor kinds, underlying resource IDs, and relative edge steps into a standardized string S_r.
6: Add S_r to candidate list $\mathcal{C}$.
7: **end for**
8: **Select** the alphabetically earliest candidate:

$$S_{\min} \leftarrow \min(\mathcal{C})$$

9: **Standardize indices**: Rename internal contracted indices to $\{i_0, i_1, i_2, \ldots\}$ and open indices to $\{o_0, o_1, \ldots\}$ according to the winning walk.
10: **Generate fingerprint**: Compute the cryptographic hash:

$$\texttt{computation_id} \leftarrow \mathrm{SHA{-}256}(S_{\min})[:20]$$

11: **return** Standardized contraction component and its `computation_id`.

Because this procedure exhaustively tests all starting tensors and deterministically selects the lexicographically minimal description, the resulting string $S_{\min}$ and its hash `computation_id` serve as a canonical fingerprint: two contraction graphs receive the same `computation_id` exactly when they are identical up to the order of their tensors and the names of their indices, with the same tensor kinds, resources and port connections. Whenever two hashes agree, the full strings are compared, so that a hash collision cannot merge different graphs. Equalities that rely on tensor symmetries or on physics relations such as Hermiticity are not detected by this structural test.

Once graphs are fingerprinted in this way, the compiler tests for graph equivalence with a single $O(1)$ hash lookup. If two graphs share the same `computation_id`, the tensor contraction is scheduled and computed only once, and the result is reused wherever needed in the global correlation matrix.

6.2.4. Reuse in a nucleon correlation matrix

Where does graph isomorphism provide practical computational reuse in realistic lattice QCD spectroscopy? Consider a 2×2 correlation matrix of spin-$1/2^+$ rest-frame nucleon interpolators constructed from two distinct diquark structures: a pseudoscalar diquark operator $\mathcal{O}_1(t)$ and an axial-vector/temporal diquark operator $\mathcal{O}_2(t)$,

$$\mathcal{O}_1(t) = \epsilon_{abc} \left[u^a(t)^\top C\gamma_5 d^b(t)\right] P_+ u^c(t), \tag{37}$$

$$\mathcal{O}_2(t) = \epsilon_{abc} \left[u^a(t)^\top C\gamma_4\gamma_5 d^b(t)\right] P_+ u^c(t). \tag{38}$$

The four matrix elements evaluated across the correlation matrix are

$$\mathbf{C}(t, t_0) = \begin{pmatrix} C_{11}(t, t_0) & C_{12}(t, t_0) \\ C_{21}(t, t_0) & C_{22}(t, t_0) \end{pmatrix}, \tag{39}$$

where each correlator element is defined by $C_{ij}(t, t_0) = \langle \mathcal{O}_i(t)\, \overline{\mathcal{O}}_j(t_0) \rangle$.

Each of the four correlators contains two Wick terms, so the matrix produces eight contraction graphs with 96 tensor occurrences in total. The global compiler pass identifies the following structure:

1. **Shared tensors across the full basis**: Because spin matrices are not part of the spatial kernels, all eight graphs use the same two baryon kernels, one built from V for the sink block and one from $V^\dagger$ for the source block, and the same up- and down-quark perambulators. Together with seven distinct spin matrices, the 96 tensor occurrences require only 11 distinct tensors, each of which is loaded or built once per time slice.
2. **Distinct contraction networks**: In this example all eight networks are distinct. In particular, the off-diagonal elements C_{12} and C_{21}, which are related by Hermiticity on the gauge ensemble, carry different spin matrices at the sink and the source ($C\gamma_5$ at the sink and the conjugate of $C\gamma_4\gamma_5$ at the source, or vice versa). The structural test of Algorithm 1 therefore treats them, correctly, as different computations. Exploiting relations of this kind would require an additional, separately validated rule and is not done.
3. **Exact algebraic cancellation (zero pruning)**: Within a single correlator sum, if two contributing scalar terms possess identical canonical components but opposite coefficients ($c_\alpha + c_\beta = 0$), SymCorr sums the coefficients and prunes the term from the execution DAG, so that no operations or buffer memory are spent on vanishing contributions.

Reuse of identical networks becomes important when the same operator structures recur across many matrix elements and Wick terms, and partial contractions shared between different networks provide a further reduction (Sec. 6). The reductions obtained for larger systems are quantified in Sec. 9.

6.3. Memory and time-slice planning
6.4. Hardware-aware execution planning
6.5. Numerical execution
7. Validation and Correctness
7.1. Validation strategy
7.2. Symbolic and Wick-contraction tests
7.3. Numerical correlator validation
7.4. End-to-end and regression testing
8. Worked Examples
8.1. Meson correlator
8.2. Baryon correlator
8.3. Multi-hadron or tetraquark example
9. Performance Evaluation
9.1. Benchmark setup
9.2. Compilation and planning overhead
9.3. Runtime performance
9.4. Memory use
9.5. Scaling
10. Software Availability and Reproducibility
10.1. Package and dependencies
10.2. User workflow
10.3. Reproducibility of the results in this work
11. Limitations and Future Development
12. Conclusions

References

[1] William Detmold, William I. Jay, Gurtej Kanwar, Phiala E. Shanahan, and Michael L. Wagman. Multiparticle interpolating operators in quantum field theories with cubic symmetry. *Phys. Rev. D*, 109:094516, May 2024.

[2] Haobo Yan, Chuan Liu, Liuming Liu, and Yu Meng. Construction of general N-body lattice operators with arbitrary momenta. *JHEP*, 10:210, 2025.

[3] Dalibor Djukanovic. Quark Contraction Tool — QCT. *Comput. Phys. Commun.*, 247:106950, 2020.

[4] R. G. Edwards, B. Joó, et al. The Chroma software system for lattice QCD. In *Lattice field theory. Proceedings, 22nd International Symposium, Lattice2004, Batavia, USA, June 21-26, 2004*, volume 140, pages 832–835, 2005.

[5] Jie Chen, Robert G. Edwards, and Weizhen Mao. Graph contractions for calculating correlation functions in lattice qcd. In *Proceedings of the Platform for Advanced Scientific Computing Conference*, PASC '23, New York, NY, USA, 2023. Association for Computing Machinery.

[6] Michael Peardon, John Bulava, Justin Foley, Colin Morningstar, Jozef Dudek, Robert G. Edwards, Balint Joo, Huey-Wen Lin, David G. Richards, and Keisuke Jimmy Juge. A Novel quark-field creation operator construction for hadronic physics in lattice QCD. *Phys. Rev. D*, 80:054506, 2009.

[7] Debsubhra Chakraborty, Piyush Srivastava, Arpith Kumar, and Nilmani Mathur. Nuclear correlation functions using first-principle calculations of lattice quantum chromodynamics. *Phys. Rev. D*, 110:114505, Dec 2024.

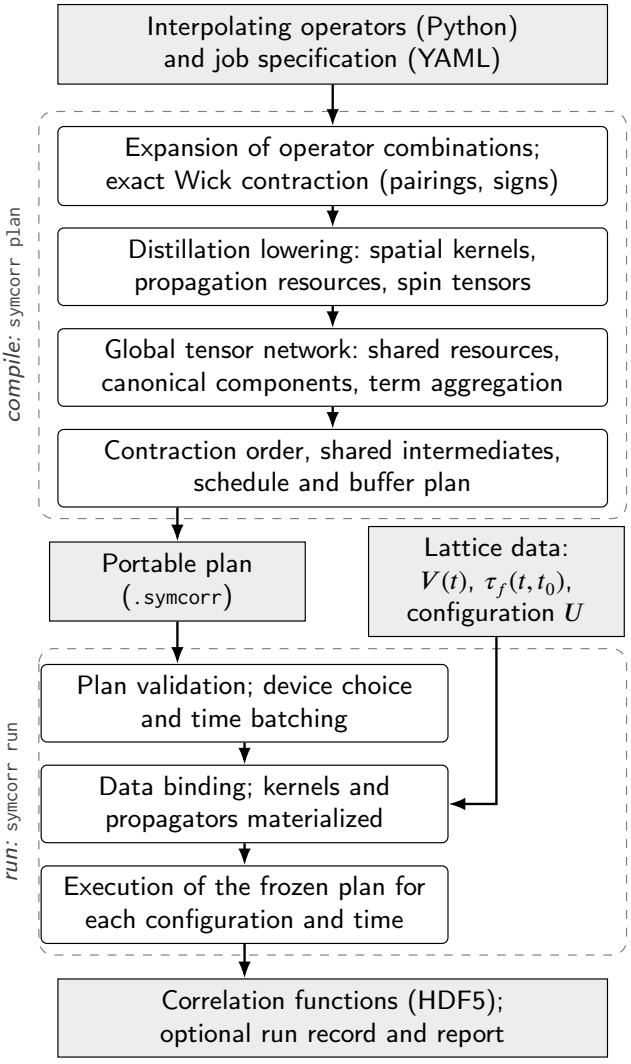
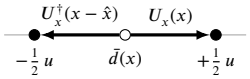


Figure 1: SymCorr workflow. The compile stage depends only on the symbolic operators and the compile settings and produces a portable, hardware-independent plan. Lattice data (Laplacian eigenvectors $V(t)$, perambulators $\tau_f(t,t_0)$ and, for derivative or displaced operators, the gauge configuration U, read as the stout-smeared spatial links used for the Laplacian eigenvectors) and the available hardware enter only at run time.

```
pion_dx = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * derivative(
        u("x", "a", "beta"), "+x"),
    momentum=(0, 0, 0),
)
```

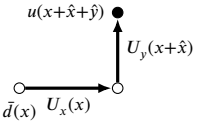
$$\mathcal{O}(t) = \sum_{\mathbf{x}} \bar{d}(x)\,\gamma_5\,(\nabla_x u)(x)$$



(a) derivative ∇_x

```
pion_txy = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * transport(
        u("x", "a", "beta"), "+x", "+y"),
    momentum=(0, 0, 0),
)
```

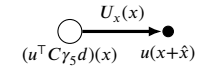
$$\mathcal{O}(t) = \sum_{\mathbf{x}} \bar{d}(x)\,\gamma_5\,U_x(x)U_y(x+\hat{x})\,u(x+\hat{x}+\hat{y})$$



(b) ordered path $(+x, +y)$

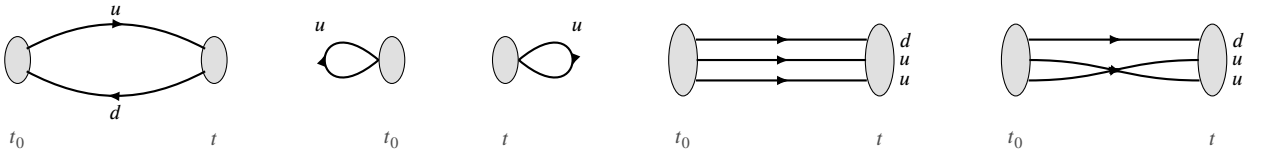
```
proton_tx = Operator(
    epsilon("a", "b", "c")
    * spin_matrix("Cg5", "alpha", "beta")
    * u("x", "a", "alpha")
    * d("x", "b", "beta")
    * transport(u("x", "c", "delta"), "+x"),
    momentum=(0, 0, 0),
)
```

$$\mathcal{O}_{\delta}(t) = \sum_{\mathbf{x}} \epsilon_{abc}\,(u_a^{\top} C\gamma_5 d_b)(x)\,\left[U_x(x)\,u(x+\hat{x})\right]_{c\delta}$$

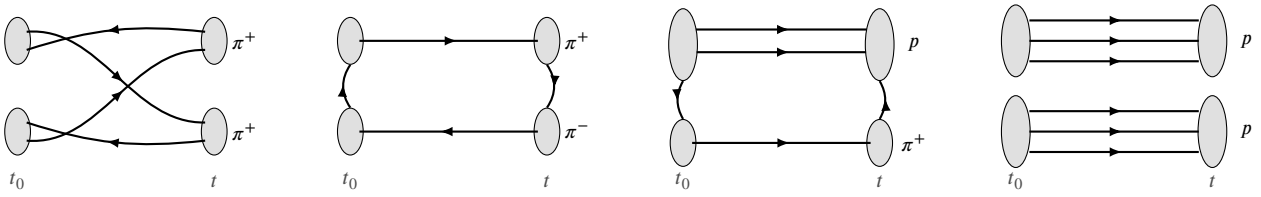


(c) spectator-displaced proton

Figure 2: Derivative and displaced operators in the SymCorr language (top), the operators they define (middle), and the lattice structure (bottom; open circles mark the position x of the undisplaced fields, filled circles the displaced quark, arrows the gauge links in the order of the path). The conventions are those of Eq. (14); U are the stout-smeared links of the gauge configuration. These three channels, together with the pion operators with ∇_y, ∇_z and the one-link transports T_{+x}, T_{+y}, T_{+z}, are validated against Chroma-LapH in Sec. 7.

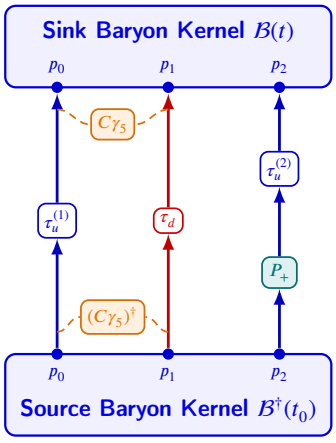


(a) meson, π^+ (b) closed loops, $\bar{u}u$ (c) baryon, direct (d) baryon, exchange

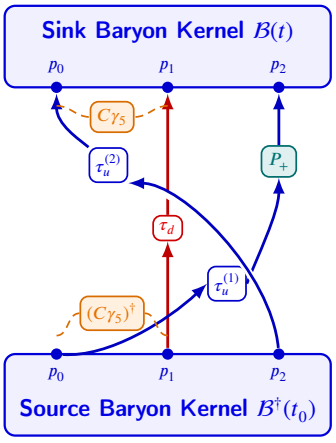


(e) two mesons, crossed u lines (f) two mesons, equal-time lines (g) baryon–meson, equal-time lines (h) two baryons, direct

Figure 3: Representative quark-flow topologies generated by SymCorr. Ellipses are hadron operators at the source time t_0 (left) and the sink time t (right); arrows follow quark propagation. (a) Single meson. (b) Disconnected term of a flavor-diagonal meson with a closed quark loop at each time. (c, d) Direct and exchange terms of the proton. (e) Two mesons with crossed lines. (f) Two mesons with one equal-time line at each time (rectangle). (g) Baryon and meson with equal-time lines. (h) Two separately projected baryons. All terms of a correlator, including the remaining exchanges, are enumerated automatically; Table 1 gives the counts.



(a) Direct contraction graph (sign +1)



(b) Exchange contraction graph (sign −1)

Figure 4: Graph representations of the two proton Wick contraction topologies in SymCorr, following the convention of distillation hadron contraction networks [5]. Blue and red solid directed lines represent up- and down-quark perambulator propagations (τ_u, τ_d) connecting source ports to sink ports. Dashed orange links represent Dirac spinor contractions with diquark matrices $C\gamma_5$ and positive-parity energy projectors P_+.